Universidad de San Carlos de Guatemala
Escuela de Ingeniería en Ciencias y Sistemas, Facultad de Ingeniería
Introducción a la programación y computación 2, 1er. Semestre 2026

SISTEMA API SAT

202400898 – Angel Adrián Rodriguez Barrientos

Resumen

El presente proyecto consiste en una solución integral de software basada en una arquitectura cliente-servidor, diseñada para simular los procesos de la Superintendencia de Administración Tributaria (SAT). El sistema está compuesto por dos componentes principales desarrollados bajo el paradigma de Programación Orientada a Objetos (POO) en lenguaje C#: una Web API (Backend) construida en .NET Core y una aplicación web (Frontend) desarrollada con Razor Pages.

Palabras clave

1. WEB
2. Modulo
3. Regex
4. JSON
5. Persistencia

Abstract

This project consists of a comprehensive software solution based on a client-server architecture, designed to simulate the processes of the Superintendencia de Administración Tributaria (SAT). The system is composed of two main components developed under the Object-Oriented Programming (OOP) paradigm in C#: a Web API (Backend) built with .NET Core and a web application (Frontend) developed using Razor Pages.

Keywords

1. WEB
2. Module
3. Regex
4. Json
5. Persistence

Introducción

En el contexto actual del desarrollo de software empresarial, la interoperabilidad y la modularidad son pilares fundamentales para la construcción de sistemas robustos y escalables. La Superintendencia de Administración Tributaria (SAT) requiere soluciones tecnológicas capaces de gestionar volúmenes significativos de transacciones electrónicas, garantizando la validez de los contribuyentes, la integridad numérica de los impuestos y la trazabilidad de la facturación comercial en el país.

El presente trabajo aborda el diseño e implementación de una solución web integral estructurada bajo una arquitectura cliente-servidor orientada a servicios. El componente central reside en un servicio backend en C# (.NET Core) estructurado bajo el paradigma de Programación Orientada a Objetos (POO), el cual actúa como una Web API encargada de procesar las solicitudes de autorización de Documentos Tributarios Electrónicos (DTE) y actualizar el Registro Tributario Unificado (RTU). Para asegurar la validez de los datos, el sistema implementa el algoritmo matemático Módulo 11 para la verificación de NITs, así como expresiones regulares (Regex) para la sanitización y extracción de marcas temporales.

Complementariamente, la capa de presentación se desarrolló utilizando el framework Razor Pages, proporcionando una interfaz limpia para la simulación de consultas, generación de resúmenes financieros, representación gráfica de datos mediante Chart.js y exportación de reportes a PDF. La persistencia de la información se gestiona localmente en archivos JSON, demostrando una alternativa ligera y funcional para el almacenamiento estructurado de datos.

Desarrollo del tema

1. Arquitectura de Software y Separación de Preocupaciones (SoC) La concepción de este sistema simulador de la Superintendencia de Administración Tributaria (SAT) requirió el diseño de una arquitectura robusta orientada a servicios, dividiendo estrictamente el entorno en dos programas aislados que se comunican mediante el protocolo HTTP. El Servicio 2, actuando como la Interfaz de Programación de Aplicaciones (API) de backend, fue desarrollado íntegramente en C# bajo el framework .NET Core. Esta decisión tecnológica permite aprovechar las características del paradigma de Programación Orientada a Objetos (POO), garantizando un diseño modular, escalable y mantenible mediante la implementación de abstracciones, encapsulamiento y polimorfismo. El Programa 1, desplegado como una aplicación web Frontend utilizando Razor Pages, asume la responsabilidad de la capa de presentación y la interacción con el usuario final, manteniendo la lógica de negocio aislada en el servidor. Esta división cliente-servidor evita el acoplamiento fuerte de los componentes lógicos y permite que el backend pueda ser consumido y testeado por clientes externos estandarizados.

2. Protocolos de Comunicación e Intercambio de Datos Estructurados El flujo masivo de información a través de la red exige un formato de serialización de datos que sea ligero, estructurado y agnóstico al lenguaje. Para este fin, se estableció el uso de JavaScript Object Notation (JSON) como el estándar de comunicación exclusivo entre el cliente y el servidor. Durante el proceso de carga de

contribuyentes para el Registro Tributario Unificado (RTU), así como en la recepción de Documentos Tributarios Electrónicos (DTE) para su autorización, el sistema mapea (deserializa) las cadenas de texto JSON hacia Data Transfer Objects (DTOs) instanciados en la memoria de C#. Este proceso asegura que la estructura de la información —que incluye variables críticas como el tiempo, referencia unívoca, NIT de los involucrados, valor comercial bruto, IVA y total neto— mantenga su integridad transaccional estricta antes de ser sometida a las reglas de validación fiscal.

3. Teoría y Aplicación del Algoritmo Matemático Módulo 11 El núcleo de la integridad tributaria simulada recae en la validación precisa del Número de Identificación Tributaria (NIT). El sistema descarta cualquier evaluación superficial e implementa computacionalmente el algoritmo Módulo 11, un estándar criptográfico ligero de suma de comprobación utilizado para detectar alteraciones numéricas. La lógica algorítmica de validación procesa cada cadena numérica siguiendo un flujo estricto: Primero, el sistema aísla el dígito verificador, que corresponde a la posición más a la derecha de la cadena (pudiendo ser un dígito entero o el carácter 'K'). Posteriormente, se itera sobre los dígitos restantes en orden inverso, multiplicando cada valor por un factor ponderado equivalente a su posición relativa (iniciando con un multiplicador de 2 para el dígito inmediato al verificador, e incrementando secuencialmente). La sumatoria total de estos productos ponderados se divide entre 11 para extraer el residuo matemático. Este residuo se resta de la constante 11; si la diferencia resultante es igual a 10, la lógica determina automáticamente que el dígito verificador legítimo debe ser la letra 'K'. En caso contrario, el remanente numérico debe coincidir exactamente con el dígito verificador proporcionado. Esta subrutina matemática se ejecuta exhaustivamente tanto para el emisor como para el receptor de cada factura, induciendo el rechazo de la petición a nivel de servidor si alguno de los actores presenta inconsistencias.

4. Sanitización Computacional Mediante Expresiones Regulares (Regex) Los sistemas informáticos de escala gubernamental se enfrentan continuamente a la variabilidad e impredecibilidad de la entrada humana. Los mensajes de solicitud de facturación presentan atributos temporales que con frecuencia incluyen ruido textual, tales como la inclusión de la palabra "Guatemala, ", nombres de otros departamentos geográficos o marcadores horarios periféricos ajenos a la marca de tiempo requerida. Para depurar esta información sin interrumpir el flujo de ejecución por excepciones de formateo, se integraron Expresiones Regulares (Regex) en la capa de procesamiento. Mediante la definición de patrones formales basados en teoría de autómatas finitos (específicamente la búsqueda y delimitación de subcadenas que satisfagan la sintaxis matemática de fecha), el algoritmo es capaz de escanear secuencias de texto contaminadas y extraer de manera aislada la fecha operativa. Esta técnica de minería y depuración de cadenas neutraliza excepciones de "Parseo" (Conversión forzada de tipos de dato) y permite que las evaluaciones matemáticas de rangos cronológicos posteriores se realicen sobre instancias de objetos DateTime robustos y validados.

5. Lógica de Negocio Financiera y Estructuración de Correlativos Tributarios Una vez que un DTE aprueba los filtros criptográficos de las identidades tributarias, el flujo de control se desplaza hacia la validación de coherencia financiera. A nivel aritmético, la aplicación verifica de forma condicional que el valor del Impuesto al Valor Agregado (IVA) sea estrictamente equivalente al producto del valor base comercial multiplicado por un factor de 0.12, aplicando una función matemática de redondeo absoluto hacia dos cifras decimales. Adyacentemente, corrobora la igualdad estricta de la ecuación donde el valor bruto sumado al impuesto debe coincidir con el campo "TOTAL" ingresado. La generación del código de autorización único representa otro de los pilares de la lógica central. La API debe sintetizar un identificador que amalgame

la marca temporal extraída bajo la máscara "yyyymmdd" concatenada con un número secuencial dinámico. Para imposibilitar colisiones de datos o duplicaciones de referencias internas, el programa orquesta un diccionario estructurado (Hash Map) en la memoria volátil que contabiliza los registros aprobados de forma atómica por día, aplicando funciones de relleno de caracteres nulos (ceros a la izquierda) para asegurar que la secuencia secuencial final cumpla con el bloque de ocho posiciones numéricas estipulado (ej. "00000001").

6. Mecanismos de Memoria Persistente y Estructuras de Datos Locales Ante el requerimiento de dotar al software de almacenamiento a largo plazo sin el acoplamiento a un motor gestor de bases de datos relacionales externo, la arquitectura recae en una persistencia gestionada directamente en el sistema de archivos del sistema operativo. Cada iteración de éxito donde el backend procesa una mutación del estado (creación de un nuevo contribuyente en el RTU, aprobación de un documento de facturación o la tabulación de anomalías), se invocan subrutinas de entrada y salida (I/O) que serializan las colecciones dinámicas y Tipos de Dato Abstracto (TDA) hacia documentos físicos de texto plano con notación JSON. Este protocolo garantiza resiliencia: frente a la terminación abrupta del ciclo de vida de la aplicación, el software es capaz de leer y reconstruir los grafos de objetos hacia la memoria RAM de manera íntegra durante la fase de inicialización de la API.

7. Trazabilidad de Anomalías, Análisis Estadístico y Renderizado Dinámico El cierre lógico del sistema involucra la explotación analítica interactiva de los datos recabados. El backend se expone a peticiones complejas donde recibe parámetros cronológicos mediante la red HTTP. El requerimiento estadístico más robusto exige la clasificación, agrupación y conteo retrospectivo de transacciones rechazadas. El sistema devuelve colecciones estructuradas que detallan cuantitativamente: fallas estructurales por algoritmo (NIT emisor o receptor inválido), ausencias registrales de datos (NIT inexistente en los diccionarios de RTU), discrepancias contables (IVA o TOTAL asimétrico) y violaciones de unicidad transaccional (referencia cruzada o duplicada). Este set de datos es formateado rígidamente por la capa web para ser presentado mediante una plantilla predefinida y estructurada al usuario.
Para el componente visual del frontend, se transiciona de la abstracción numérica hacia métricas de inteligencia empresarial. Mediante inyecciones asíncronas de JavaScript y el uso de librerías especializadas (Chart.js), se construyen proyecciones cartesianas: gráficos de barras que discriminan los volúmenes de IVA fiscalizado por actor, y diagramas de vectores secuenciales para la lectura temporal del impacto económico en un rango de días seleccionado. Finalmente, la plataforma concluye su función operativa al empalmar la inyección de estas métricas gráficas y numéricas con los motores nativos de impresión de los navegadores web modernos, ensamblando documentos formales estáticos y exportándolos exitosamente como ficheros PDF estandarizados, cerrando así la cadena del proceso tecnológico de simulación fiscal.

Conclusiones

La arquitectura cliente-servidor distribuida, compuesta por un Backend en .NET Core (Web API) y un Frontend en Razor Pages, demostró ser una solución altamente eficiente e independiente para la simulación de servicios tributarios, garantizando un desacoplamiento efectivo entre la lógica de negocio y la capa de presentación.

La implementación computacional del algoritmo matemático Módulo 11 aseguró la validación estricta del Número de Identificación Tributaria (NIT) para emisores y receptores, garantizando que únicamente las entidades fiscales legítimas puedan registrar y

procesar Documentos Tributarios Electrónicos (DTE).
El uso de Expresiones Regulares (Regex) en el servidor permitió la correcta extracción y sanitización de las marcas temporales presentes en los mensajes de entrada, eliminando el ruido de texto secundario y asegurando la integridad de los objetos de fecha sin interrumpir la ejecución del sistema por excepciones de formateo.

El mecanismo de persistencia basado en archivos JSON locales proveyó una estrategia de almacenamiento ligera, funcional y resiliente, permitiendo la reconstrucción del estado de la aplicación en memoria y asegurando la trazabilidad de los datos aprobados y de las anomalías registradas.

La integración de herramientas de visualización dinámica como Chart.js y la función nativa de impresión a PDF en el Frontend permitió la consolidación analítica de la información fiscal, facilitando la interpretación visual de resúmenes financieros, reportes de errores y estadísticas por rango de fechas.

Referencias bibliográficas

Microsoft Corporation. *Documentación oficial de .NET Core: Creación de Web APIs mínimas en C#.* Microsoft Learn, 2024. [En línea]. Disponible en: [https://learn.microsoft.com/dotnet/architecture/microservices/] (https://learn.microsoft.com/dotnet/architecture/microservices/)

Microsoft Corporation. *Introducción a Razor Pages en ASP.NET Core*. Microsoft Learn, 2024. [En línea]. Disponible en: [https://learn.microsoft.com/aspnet/core/razor-pages/] (https://learn.microsoft.com/aspnet/core/razor-pages/)

Anexos

Diagrama de clases

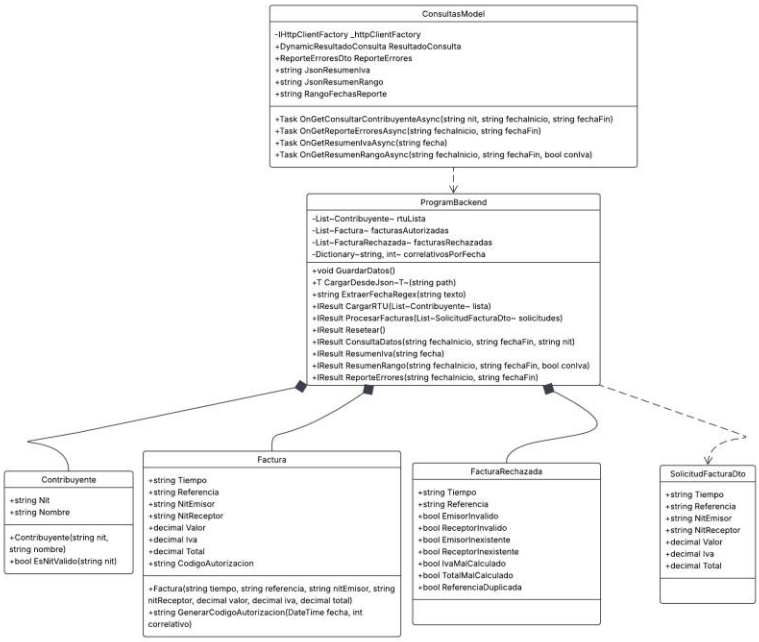


Diagrama de actividades

Semana	Actividad
Semana 1	Configuración de la solución cliente-servidor, creación de proyectos en .NET Core (Web API BackendSAT y Razor Pages FrontendSAT) y diseño de las clases base del dominio (Contribuyente, Factura, FacturaRechazada).

Semana 2	Implementación del algoritmo matemático de validación de NITs por Módulo 11, lógica de extracción de fechas mediante Expresiones Regulares (Regex) y desarrollo de la capa de persistencia mediante archivos JSON (rtu.json, facturas.json, errores.json).
Semana 3	Desarrollo de los endpoints de procesamiento de facturas con generación de correlativos diarios (yyyymmdd########), cálculos de IVA/Total y creación de las vistas de carga de archivos en Razor Pages.
Semana 4	Implementación de las consultas avanzadas, reportes estructurados de errores por rango de fechas, renderizado de gráficas dinámicas con Chart.js, exportación nativa a PDF y pruebas de integración end-to-end.